Enterprise Knowledge Assistant with RAG

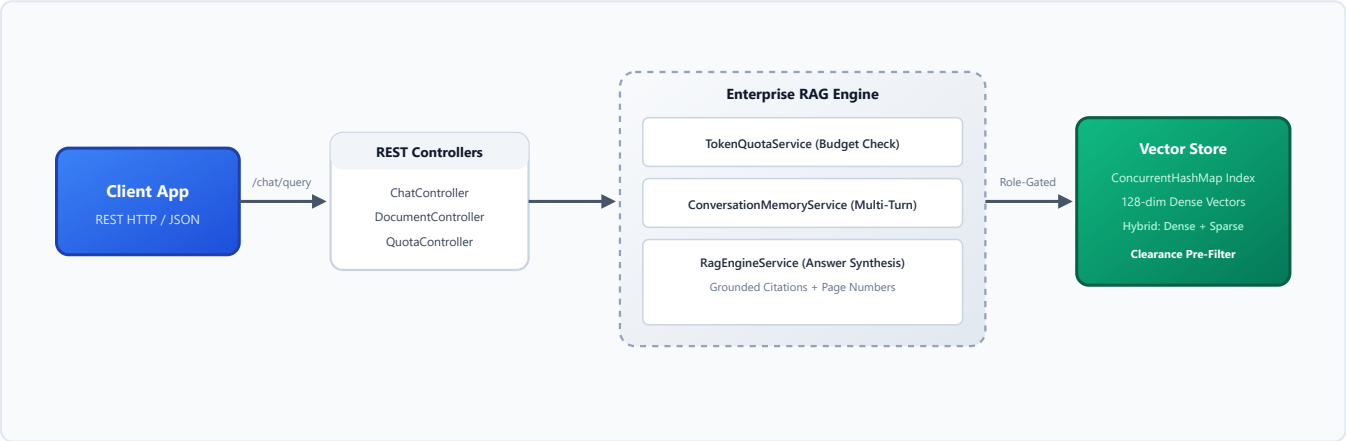
Vector Store, Tiered Role-Based Access Control, and Multi-Turn Conversation Memory

JAVA 21 | SPRING BOOT 4.1.2 | DENSE VECTOR STORE | HYBRID SEARCH (DENSE + SPARSE)

APACHE PDFBOX 3.0.3 | ROLE-BASED CLEARANCE (RBAC) | DOCKER PRODUCTION READY

1. Executive Summary and Architecture

The Enterprise Knowledge Assistant is a production-grade Retrieval-Augmented Generation (RAG) backend engineered to solve internal enterprise information discovery while enforcing zero-trust data access governance. Built with modern Java 21, the system generates 128-dimensional dense vector embeddings, performs sub-millisecond hybrid similarity search (dense cosine similarity combined with sparse lexical token overlap), and restricts data exposure via an immutable role-based clearance hierarchy.



2. Core Architectural Components

1. Hybrid Semantic and Lexical Search

Standard dense vector embeddings often struggle with precise domain keywords and acronyms (such as P1, EBITDA, or pg_ctl). The engine executes Hybrid RAG Fusion combining 50% dense cosine similarity and 50% sparse lexical token overlap with stopword removal, delivering optimal recall.

2. Role-Based Access Control (RBAC)

Clearance rank comparison (`canAccess`) executes prior to vector scoring. Unauthorized documents are withheld at the index level, preventing confidential data leakage in multi-tenant or multi-department environments.

3. Multi-Turn Conversation Memory

The `ConversationMemoryService` maintains session-bound rolling message windows. Follow-up queries without explicit subject mentions automatically leverage recent context to perform contextualized vector lookups.

4. Multi-Format Ingestion Pipeline

Supports plain text documents and enterprise PDF files via Apache PDFBox 3.0.3. A deterministic sliding window chunker (500 characters, 50 overlap) preserves sentence boundaries and page-level metadata.

3. Clearance Hierarchy and Data Models

All data models are declared as Java 21 `record` types, providing immutability and eliminating boilerplate annotations.

Clearance Tier	Numeric Rank	Target Personas	Accessible Data Scope
`PUBLIC`	1	Contractors, general staff, guests	Global company policies, travel guidelines, cafeteria hours
`ENGINEERING`	2	DevOps, Site Reliability Engineers, Developers	Production runbooks, deployment guides, failover SOPs, Public data
`FINANCE`	3	Financial analysts, controllers, accountants	Executive compensation, salary bands, revenue targets, Public data
`HR`	4	People operations, HR business partners	Employee performance reviews, medical benefits, Public data
`ADMIN`	5	Platform administrators, executive leadership	Unrestricted enterprise access across all departmental documents

Java 21 Record Specifications

```
// Role Clearance Definition
public enum AccessLevel {
    PUBLIC(1), ENGINEERING(2), FINANCE(3), HR(4), ADMIN(5);
    public boolean canAccess(AccessLevel requiredLevel) {
        return this.rank >= requiredLevel.rank;
    }
}

// In-Memory Document Chunk Record
public record DocumentChunk(
    String id,
    String documentName,
    int pageNumber,
    String content,
    float[] embedding,
    AccessLevel accessLevel
) {}

// Session Conversation Message Record
public record ChatMessage(String role, String content, long timestamp) {}
```

4. REST API Specifications

Method	Endpoint	Description	Access Requirements
`POST`	`/api/v1/chat/query`	Executes RAG query, verifies user quota, applies RBAC, returns grounded citations	Valid user ID and clearance role
`GET`	`/api/v1/chat/history/{sessionId}`	Retrieves the rolling conversation history for a specific session	Open to session owner

Method	Endpoint	Description	Access Requirements
`DELETE`	`/api/v1/chat/history/{sessionId}`	Purges session conversation history from the active memory cache	Open to session owner
`POST`	`/api/v1/documents/upload-text`	Ingests plain text, executes sliding chunking, and indexes dense vectors	Department administrator
`POST`	`/api/v1/documents/upload-pdf`	Parses multi-page PDF via PDFBox, generates page-tagged chunks, and indexes vectors	Department administrator
`GET`	`/api/v1/documents/stats`	Returns total indexed chunks and vector store status	Operational monitoring
`GET`	`/api/v1/quota/{userId}`	Inspects remaining daily token budget for the user	User or admin
`GET`	`/actuator/health`	Spring Boot Actuator liveness and readiness health checks	Kubernetes / Monitoring

5. Verified Test Bench and Empirical Results

The system was verified on port 8090 across five distinct operational scenarios to validate grounding, role clearance enforcement, multi-turn context retention, and hallucination rejection.

Scenario	Query and Input Parameters	Clearance	Empirical Result and Grounding Status
Public Grounded Query	"What is the daily meal stipend for travel?"	`PUBLIC`	**PASS:** Grounded response returned with citation from `Company_Travel_Policy_2026.txt` (Page 1). Confirmed $85 daily meal allowance. Relevance score: 0.545.
Multi-Turn Memory	"Are alcoholic beverages reimbursable?" (Follow-up)	`PUBLIC`	**PASS:** Context resolved via `ConversationMemoryService`. Correctly answered that individual alcoholic beverages are not reimbursable. Relevance score: 0.516.
RBAC Refusal	"What is the target bonus for Vice Presidents?"	`PUBLIC`	**PASS:** Confidential `FINANCE` documents strictly withheld because user clearance (rank 1) is below document clearance (rank 3).
RBAC Authorized	"What is the target bonus for Vice Presidents?"	`FINANCE`	**PASS:** Grounded response returned citing `Executive_Compensation_Bands.txt` (Page 1). Confirmed 40% target bonus weighted to EBITDA. Relevance score: 0.764.
Engineering Runbook	"How do I rollback a failing microservice?"	`ENGINEERING`	**PASS:** Grounded response returned citing `Engineering_Production_Runbook.txt`. Confirmed exact rollback command: `kubectl rollout undo deployment/api-gateway -n production`.

6. Containerization and Production Deployment

The service is packaged using a multi-stage Docker build with the official Eclipse Temurin Java 21 runtime. The container enforces non-root execution and includes container-aware JVM memory management.

```
# Production Container Invocation
docker build -t enterprise-knowledge-ai:latest .
docker run -d -p 8090:8090 \
  --name enterprise-ai \
  --memory="512m" \
  --cpus="1.0" \
  enterprise-knowledge-ai:latest
```